2026 IEEE/ACIS 24th International Conference on Software Engineering Research, Management and Applications (SERA) | 979-8-3195-0657-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/SERA69989.2026.11618631

A⁴-Gov: A Governance Architecture for AI Service Systems

Min Seon Ahn

Graduate School, Department of IT Engineering, Sookmyung Women's University

Seoul, South Korea

msahn@sookmyung.ac.kr

Yong Ik Yoon(*Corresponding Author)

Graduate School, Department of IT Engineering, Sookmyung Women's University

Seoul, South Korea

yiyoon@sookmyung.ac.kr

Abstract—Public AI services often undergo ex ante review before deployment, but the resulting risk decision must still be carried into operation through concrete controls, review triggers, and evidence records. This paper defines A⁴-Gov as a runtime governance architecture for AI service systems that performs that operational role. The architecture assumes that review or risk grading has already determined the control level required for the service or case. A⁴-Gov then carries that decision into four runtime functions: Awareness, which derives the active baseline from policy and service context; Assessment, which identifies governance-relevant deviation from that baseline; Adaptability, which selects bounded intervention and triggers human review when required; and Accountability, which preserves policy state, intervention rationale, and audit-ready evidence. The paper positions the architecture against AI governance frameworks and self-adaptive software architectures, then examines it through a public-service walk-through and a failure-oriented analytical check. The contribution is architectural: it specifies how review decisions, bounded intervention, and accountability logging remain connected during operation.

Keywords - AI governance, AI service systems, public-service AI, software architecture, self-adaptive systems, auditability, human escalation, accountability

I. Introduction

Artificial intelligence is increasingly used to structure operational decisions in service environments. Public-service settings provide a useful example because questions of safety, fairness, rights, and administrative responsibility become visible quickly. As these systems become more deeply embedded in service workflows, their quality cannot be assessed only in terms of predictive accuracy or throughput. Their behavior must also remain governable, reviewable, and accountable during operation.

This challenge has intensified recent work on AI governance across ethics, regulation, public administration, and standards. Prior studies have significantly advanced the field by articulating normative principles, legal duties, organizational controls, and audit mechanisms [1]-[8]. Yet much of this literature still focuses on what should be governed rather than on how policy state, review conditions, and accountability records should function inside the decision loop of an AI-enabled service system.

The software-engineering literature on self-adaptive systems helps frame that gap more precisely. Feedback loops, architecture-based self-adaptation, and autonomic computing have shown how systems can observe state, analyze deviation, select responses, and execute change under explicit control structures [13], [14], [16]. However, those architectures are typically optimized for performance, resilience, or quality-of-service objectives. They do not directly explain how policy baselines, rights-sensitive thresholds, human escalation, and accountability traces should be modeled as first-class governance objects in public-service AI.

A⁴-Gov is introduced here as the runtime governance layer that follows review and risk-grading decisions. Once a service or case has been assigned a control level, the architecture carries that decision into operation through baseline formation, deviation interpretation, bounded intervention, human escalation, and accountability logging. A⁴-Gov is therefore not a separate ethics framework; it is the operational mechanism that preserves control requirements after deployment begins.

This paper makes three concrete contributions. It defines A⁴-Gov as a runtime governance architecture with explicit control objects, state variables, escalation conditions, and audit artifacts. It locates the architecture after review and risk-grading decisions by showing how policy state is carried into baseline formation, bounded response, and evidence logging during operation. It then examines the architecture through a public-service walk-through and a failure-oriented check that test explicit policy state, governance-relevant deviation, bounded intervention, and review-ready evidence.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:22:13 UTC from IEEE Xplore. Restrictions apply.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 clarifies the conceptual distinction between A⁴-Gov and existing governance frameworks. Section 4 defines the architecture and its design constraints. Section 5 reads the stages through a public-service example and failure-oriented check. Section 6 discusses implications and limitations. Section 7 concludes.

II. Related Work

Four research streams frame the proposed architecture: principle-centered AI governance, institutional and standards-based governance frameworks, operational accountability mechanisms, and self-adaptive software architectures.

A. Principle-Centered AI Governance

The first stream focuses on the normative foundations of AI governance, including fairness, accountability, transparency, responsibility, and non-maleficence. This literature has been essential for clarifying why AI deployment must remain aligned with social and legal values, especially in high-stakes domains [1]-[3]. However, principle-centered work often leaves unresolved the engineering question of how such principles become executable constraints in real-time operation.

This limitation is visible in the literature on AI ethics guidelines. Jobin, Ienca, and Vayena show that AI ethics guidelines converge around recurring principles but diverge in implementation [1]. Hagendorff likewise argues that ethics guidelines are normatively valuable but often weak in enforcement [2]. Mittelstadt et al. further show that the debate around algorithmic ethics becomes most consequential when algorithmic systems shape institutional outcomes, yet the connection between normative expectation and concrete system behavior often remains underspecified [3].

B. Institutional and Standards-Based Governance Frameworks

The second stream emphasizes organizational and regulatory governance. The NIST AI Risk Management Framework defines organization-level risk functions and governance processes [4]. The NIST Generative AI Profile extends that logic to risks that are more specific to generative-AI systems [5]. The EU AI Act specifies regulatory obligations for providers and deployers [6], while IEEE 7000 defines a process for addressing ethical concerns during system design [7]. These frameworks are highly valuable because they identify governance responsibilities, risk categories, compliance expectations, and oversight structures.

These frameworks establish what responsible governance should cover. They help organizations govern AI systems, but they do not fully specify how policy-aligned operating standards should be represented and enforced during system execution. Their dominant level of analysis remains the institution, the lifecycle process, or the compliance regime rather than the operating control loop of an AI service.

III. Accountability, Auditability, and Adaptive Governance

The third stream addresses mechanisms such as documentation, explainability, auditing, and adaptive governance. Mokander and Floridi show that operationalizing AI governance through ethics-based auditing requires concrete integration with organizational evidence flows rather than ethical declarations alone [8]. Taeihagh argues that rapidly evolving AI capability requires adaptive governance [9]. Reuel and Undheim make a similar claim in the narrower context of generative AI, where governance frameworks must support continuous revision under changing capability profiles [10]. Koenig and Wenzelburger add a public-sector perspective, showing that algorithmic decision-making can produce a legitimacy gap when administrative action becomes detached from accountable public reasoning [11]. Model cards improve structured reporting and downstream review, but they remain primarily documentation mechanisms rather than internal control structures [12].

A. Self-Adaptive Architectures

The fourth stream comes from software engineering rather than AI governance. Kephart and Chess introduced autonomic computing as a vision of systems that monitor, analyze, plan, and execute adaptation under shared knowledge [13]. Cheng et al. and Brun et al. argue that control loops should become first-class entities in the engineering of self-adaptive software [14], [15]. Garlan, Schmerl, and Cheng show how architecture-based self-adaptation can use system models, evaluation logic, and adaptation managers to maintain system goals under changing conditions [16].

This lineage is directly relevant to the present paper because it provides the architectural vocabulary for observation, control, and adaptation during operation. At the same time, these works do not primarily address governance-specific concerns such as rights-sensitive escalation, immutable policy constraints, or audit-ready accountability traces. A⁴-Gov draws on the software-architecture logic of adaptation while shifting the adaptation target from generic quality optimization to policy-aware governance.

979-8-3195-0657-3/26/$31.00 ©2026 IEEE

SERA 2026, May 27-29, 2026, Towson, USA
Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:22:13 UTC from IEEE Xplore. Restrictions apply.

Table 1. Comparative positioning of A⁴-Gov against existing governance and adaptive-control frameworks

Framework	Analytical level	Governance or control object	Operational timing	Primary evidence
NIST AI RMF	organization and lifecycle governance	risk processes and controls	pre-deployment and ongoing management	risk documentation, control artifacts
EU AI Act	legal and regulatory governance	compliance obligations and provider duties	lifecycle compliance	conformity evidence, technical documentation
A⁴-Gov	governance architecture	policy baseline, deviation, bounded action, escalation, accountability	continuous operation	deviation logs, action traces, revised baselines, escalation records

B. Research Gap

Taken together, the literature provides strong guidance on what should be governed and how governance responsibilities should be organized. What remains underdeveloped is the operational layer that follows review and risk-assignment decisions. Governance frameworks define responsibilities and control expectations, but they do not specify how a running AI service should preserve policy state, enforce bounded intervention, or write review-ready evidence once deployment begins. A⁴-Gov addresses that narrower problem. It functions as the runtime governance layer that carries control requirements into operation. Table 1 therefore contrasts lifecycle governance frameworks with the runtime control objects preserved by A⁴-Gov.

The central distinction is not simply internal versus external governance. It is that A⁴-Gov links policy state, deviation, bounded action, escalation, and evidence as runtime objects after review decisions have already been made.

IV. Conceptual Distinction from Existing Governance Frameworks

Existing governance frameworks are typically designed around oversight, compliance, risk management, and institutional accountability. A⁴-Gov operates at a different layer. It begins after review obligations and control requirements have been specified and turns those requirements into runtime state, bounded actions, review triggers, and evidence records within an AI service system.

This distinction matters because the architecture does not compete with lifecycle governance frameworks. It carries their control expectations into operation. In A⁴-Gov, policy requirements enter the loop as active policy state, baseline conditions, intervention limits, and review obligations that remain inspectable during execution [4]-[8].

A⁴-Gov is also related to feedback-loop and self-adaptive architectures [13], [14], [16], but the target of control is different. Existing adaptive architectures provide monitoring and response logic. A⁴-Gov adds a governance-specific chain from policy context to deviation interpretation, bounded intervention, human escalation, and review-ready evidence.

V. *A⁴-Gov Architecture*

This section defines A⁴-Gov as the runtime layer that executes governance requirements after review and control conditions have been set. The architectural task is to preserve policy state during operation, detect governance-relevant deviation, select bounded action, and retain review-ready evidence.

A. Components and State Ownership

A⁴-Gov can be represented as eight cooperating architectural components: a Policy Registry, a Context Normalizer, a Baseline Generator, a State Monitor, a Deviation Analyzer, a Decision Manager, an Escalation Manager, and an Audit Logger. The Policy Registry owns the current policy snapshot and the distinction between immutable constraints and adjustable thresholds. The Baseline Generator owns the active baseline B_t. The Deviation Analyzer owns the deviation record D_t. The Decision Manager proposes a_t*, while the Escalation Manager governs H_t and determines whether automated continuation is allowed. The Audit Logger writes L_t as the durable review artifact.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:22:13 UTC from IEEE Xplore. Restrictions apply.

The architecture assumes that upstream review, service approval, or risk grading has already assigned a control profile to the service or case. That control profile enters the loop through the Policy Registry together with the active policy snapshot. The runtime task is then to preserve those requirements during baseline generation, deviation analysis, bounded response selection, human escalation, and accountability logging. Each stage leaves an inspectable artifact: Θ_t, X_t, B_t, P_t, D_t, a_t*, H_t, and L_t.

B. Notation and Design Constraints

The following notation is used as a formal shorthand to identify state variables and their dependencies, not as a complete computational specification.

The architecture uses the following state variables: X_t, Θ_t, B_t, P_t, D_t, W_t, a_t*, H_t, B_{t+1} , and L_t.

In A^4 -Gov, the baseline is not a purely statistical reference point. It is a policy-aligned operating standard derived from service rules, fairness thresholds, safety constraints, and procedural requirements. The architecture separates immutable governance constraints from tunable operational baselines. Legal prohibitions, mandatory safety caps, and rights-sensitive review rules are immutable during system operation. Service-level thresholds, monitoring sensitivity, and batch-level routing rules may be revised only within bounded policy authority. This distinction prevents the recalibration loop from normalizing unsafe or unlawful drift.

C. A1 Awareness

Awareness identifies the policy baseline relevant to the current operational context. It interprets the normalized input together with the active policy state:

$$\boldsymbol{B_t = f_{(aw)}(X_t, \Theta_t)} \tag{1}$$

The role of this stage is not yet to judge observed performance. Its task is to determine what should count as an acceptable state under the active governance context. The output of Awareness is therefore a policy-aligned baseline that serves as the reference point for later evaluation.

D. A2 Assessment

Assessment evaluates the difference between the observed system state and the policy-aligned baseline:

$$\boldsymbol{D_t = g_{(as)}(P_t, B_t)} \tag{2}$$

Here, D_t encodes governance-relevant properties such as severity, persistence, fairness impact, and procedural breach, rather than functioning as a generic error score. Once the gap is explicit, governance becomes a concrete basis for intervention and review.

E. A3 Adaptability and Human Escalation

Adaptability selects a response when the observed system state deviates from the governance baseline. Because governance objectives may conflict, this stage uses weighted priorities to determine which deviations should be minimized first:

$$\boldsymbol{a_t^* = arg\,min_a \sum_i w_{(i,t)}\, d_i(a)} \tag{3}$$

where $w_{i,t}$ represents the priority assigned to governance objective i at time t, and $d_i(a)$ represents the residual deviation associated with candidate action a.

The architecture does not permit unrestricted self-redefinition. Recalibration is bounded:

$$\boldsymbol{B_{(t+1)} = h(B_t, D_t, W_t, a_t^*)} \tag{4}$$

The function h may update tunable operating thresholds, routing rules, or monitoring sensitivity, but it may not alter immutable legal or safety constraints held by the Policy Registry. Human approval is mandatory when baseline revision would change rights-sensitive handling, reduce safety margins, or resolve unresolved policy conflict.

Human escalation is therefore a first-class subsystem rather than an afterthought. In A^4 -Gov, H_t becomes active when at least one of three conditions holds: the deviation exceeds a predefined safety or fairness threshold; the case affects rights-sensitive or vulnerable populations; or the proposed recalibration crosses a policy authority boundary. When H_t is active, the Escalation Manager pauses automatic continuation for the affected case or batch, routes the case to the authorized office, records the reviewer identity and decision, and writes the approved action back into the loop as part of L_t. This design is consistent with work on meaningful human control, which emphasizes that responsibility must remain aligned with human ability and authority to intervene [17].

F. A4 Accountability

Accountability records the rationale for decisions, the basis of weighting, the presence or absence of human escalation, and the sequence of operational Adaptability:

$$\boldsymbol{L_t = \{X_t, \Theta_t, B_t, D_t, a_t^*, H_t, B_{(t+1)}\}} \tag{5}$$

For review and audit in public-service settings, the most important artifacts are the input context, the active policy standard, the measured deviation, the selected adaptability or escalation decision, and the revised baseline with its rationale. These records form the evidence core of A^4 -Gov: they carry review requirements into operational traces that can later support explanation, institutional review, and accountability.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:22:13 UTC from IEEE Xplore. Restrictions apply.

Table 2 summarizes the example as review-ready evidence.

Artifact	Example value and validation role
$\mathbf{X_t}$	public_safety, high-risk location, vulnerable area; establishes the operational context
$\mathbf{\Theta_t}$	P1 safety rule, fairness guard, 2h cap, mandatory review rule; establishes active policy state
$\mathbf{B_t}$	priority = P1, max_response = 2h, fairness_guard = enabled; shows policy-aligned baseline formation
$\mathbf{P_t}$	priority = P3, estimated_response = 18h, no fairness alert; shows observed system behavior
$\mathbf{D_t}$	critical safety gap, persistent fairness gap; shows governance-relevant deviation
$\mathbf{a_t^*}$	override to P1, open batch review, require supervisor confirmation; shows bounded intervention
$\mathbf{H_t}$	true, municipal safety office review required; shows human escalation logic
$\mathbf{B_{t+1}}$	tighter clustering alert threshold, unchanged statutory cap; shows bounded recalibration
$\mathbf{L_t}$	case id, policy snapshot id, baseline version, reviewer id, rationale code; shows audit-ready accountability evidence

G. Architectural Summary

Together, these four functions form the operational governance layer that follows review and risk-assignment decisions. Figure 1 shows the runtime loop; Figures 2 and 3 can be read together through the evidence core preserved in L_t. Policy state enters through Θ_t and B_t, bounded intervention passes through a_t^* and H_t, and accountability is closed through the recorded trail. This is the architectural basis for treating A^4-Gov as more than a general operating concept.

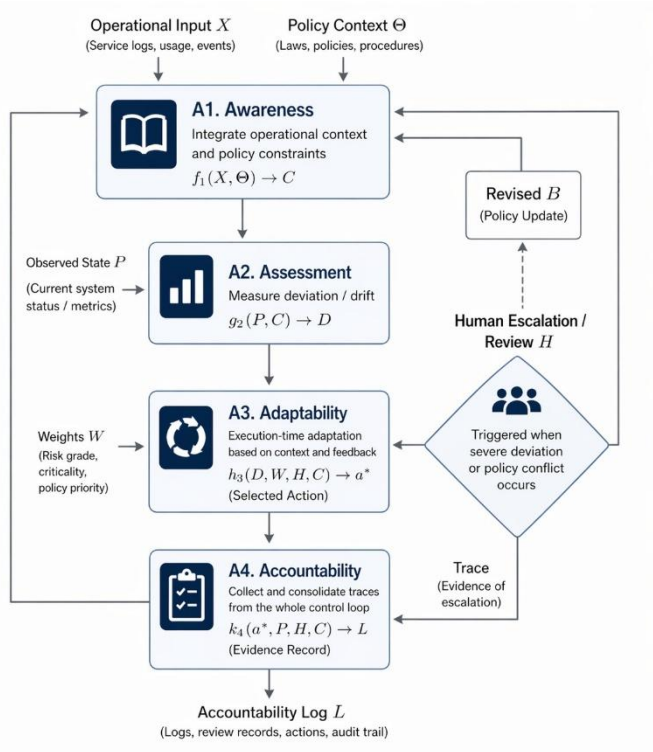


Figure 1. A⁴-Gov as a runtime governance loop with bounded recalibration and accountability logging

VI. Reading the Stages Through a Public-Service Example

This section provides an analytical walk-through of the architecture in a public-service setting. The question is whether the design preserves four required properties: explicit policy state, governance-relevant deviation, bounded intervention, and review-ready evidence.

The discussion is analytical rather than comparative. It tests whether the defined components and state variables are sufficient to preserve those four properties in the example.

A. Architecture Evaluation Design

We examine four questions. EQ1 asks whether policy state remains explicit in the operating loop. EQ2 asks whether the architecture turns observation into governance-relevant deviation rather than generic anomaly scoring. EQ3 asks whether bounded intervention and human escalation remain tied to explicit authority limits. EQ4 asks whether the log preserves enough state for retrospective review and institutional accountability.

These questions are checked against observable artifacts. EQ1 depends on Θ_t, B_t, and baseline versioning; EQ2 on P_t and D_t; EQ3 on a_t^*, constraint flags, and H_t; and EQ4 on L_t, reviewer identity, rationale codes, and revision records.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:22:13 UTC from IEEE Xplore. Restrictions apply.

B. Illustrative Public-Service Example

Consider a municipal AI-assisted public-service triage system that processes citizen complaints, welfare requests, and safety-related reports. A specific case arrives at time t: an urgent public-safety-related request from a vulnerable district that has already been reported more than once. The normalized operational input X_t includes service_type = public_safety, location_risk = high, vulnerable_area = true, and reported_delay = repeated.

The active policy context Θ_t contains four relevant rules: urgent safety-related requests should receive priority level P1; requests affecting vulnerable districts must not be assigned lower priority solely because of historically lower response rates; any predicted response window above two hours for a P1 case is a procedural breach; and any batch-level fairness deviation involving a vulnerable district requires human review by the responsible municipal office.

From this context, A1 derives the baseline B_t: required_priority = P1, max_response_window = 2h, fairness_guard = enabled, and mandatory_review_if_batch_bias = true. The observed system state P_t, however, shows that the model assigned the case P3, estimated a response window of 18 hours, and did not raise a fairness alert because the model treated district-level under-service as historical normality rather than active deviation.

A2 therefore generates a deviation record D_t with safety_gap = critical, response_window_breach = true, and district_fairness_gap = persistent. The priority vector W_t for this service window weights safety at 0.50, fairness at 0.30, and procedural regularity at 0.20. Under these weights, A3 proposes the bounded action a_t^* = {override priority to P1, open batch review for similar district cases, require supervisor confirmation}. Because the proposed action concerns both a rights-sensitive service context and a persistent district-level fairness pattern, the Escalation Manager activates H_t = true and routes the case batch to the municipal safety office.

The crucial architectural point is that B_{t+1} does not rewrite the governing law. The revised baseline tightens the monitoring threshold for clustered safety complaints from vulnerable districts and adds a mandatory fairness alert for the next batch, but it does not alter the immutable P1 requirement or the statutory response cap. A4 writes L_t with the case identifier, timestamp, policy snapshot id, baseline version, deviation category, selected action, escalation status, reviewer office, and rationale code.

C. Structural Findings and Comparator

The example preserves the required properties as follows. Policy state remains explicit through Θ_t and B_t. Governance-relevant deviation appears in D_t rather than as a generic anomaly flag. The intervention path remains bounded because a_t^* and H_t are conditioned by policy limits and review authority. The resulting log retains the baseline, deviation, action rationale, reviewer authority, and revised baseline as structured evidence.

The comparator clarifies the role of A^4-Gov. A static-threshold design might flag the low-priority assignment, but it would not preserve the policy snapshot, the fairness rationale, or the authority boundary that triggered review. An external-audit-only design could inspect the incident later, but it would not intervene while the rights-sensitive prioritization was unfolding. A^4-Gov joins runtime control with post-deployment evidence.

D. Failure-Oriented Check

The same example can be stressed with a failure condition. Suppose the system detects the deviation but revises the future baseline by lowering the safety alert threshold for that district in order to reduce apparent false positives. If immutable constraints and approval boundaries are not separated, the architecture would normalize harmful drift instead of correcting it. A^4-Gov treats this as a governance failure. The recalibration is rejected, H_t remains active, and the Audit Logger records that the proposed revision exceeded operating authority. In that sense, the escalation path serves as an assurance mechanism: it prevents unsafe continuation while preserving a bounded fallback path for review [18].

This stress case clarifies the paper's narrower adequacy claim. A governance-oriented loop must expose the active policy baseline, interpret deviation in governance terms, constrain recalibration, and preserve enough trace data for institutional review.

VII. Discussion

A^4-Gov is best read as an operational governance layer rather than as a separate ethics framework. Governance failures in public AI services often appear after deployment, when changing conditions, undocumented overrides, or conflicting priorities separate formal policy from actual system behavior. The architecture addresses that gap by keeping policy state, intervention limits, review triggers, and evidence records inside the runtime loop.

This is particularly important in public-service settings, where institutions must justify not only outputs but also changes in risk, priority, and policy interpretation over time. The combination of baseline formation, deviation analysis, bounded intervention, human review, and accountability logging provides a concrete operating mechanism for that task.

The scope remains bounded. The paper does not compare performance against alternative control models, and the notation is descriptive rather than control-theoretic proof. What it provides is a design that later work can instrument with explicit grading rules, domain thresholds, and real operational logs.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:22:13 UTC from IEEE Xplore. Restrictions apply.

VIII. CONCLUSION

A^4-Gov is presented in this paper as a runtime governance architecture for AI service systems. Its role is not to replace lifecycle governance frameworks but to carry their control requirements into operation through four linked functions: Awareness, Assessment, Adaptability, and Accountability. Within that loop, policy state, deviation, bounded intervention, human escalation, and evidence logging are treated as explicit design objects.

The public-service walk-through shows the intended use of the architecture. Once review conditions and risk grades have been assigned, A^4-Gov preserves them during operation and records the resulting decision trail. That linkage between policy state, bounded response, and evidence is the paper's main claim. Future work can extend the architecture with explicit grading functions, domain-specific thresholds, and empirical testing with real service logs.

Acknowledgment

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP)-ICAN (ICT Challenge and Advanced Network of HRD) grant funded by the Korea government (Ministry of Science and ICT) (IITP-2026-RS-2022-00156299). The figures were conceptually designed by the authors.

ChatGPT was used only to improve the visual presentation of the final figure images. All technical content, figure labels, captions, and final revisions were reviewed and approved by the authors.

References

[1] A. Jobin, M. Ienca, and E. Vayena, "The global landscape of AI ethics guidelines," Nature Machine Intelligence, vol. 1, pp. 389-399, 2019, doi: 10.1038/s42256-019-0088-2.

[2] T. Hagendorff, "The Ethics of AI Ethics: An Evaluation of Guidelines," Minds and Machines, vol. 30, pp. 99-120, 2020, doi: 10.1007/s11023-020-09517-8.

[3] B. D. Mittelstadt, P. Allo, M. Taddeo, S. Wachter, and L. Floridi, "The ethics of algorithms: Mapping the debate," Big Data and Society, vol. 3, no. 2, 2016, doi: 10.1177/2053951716679679.

[4] E. Tabassi, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2023, doi: 10.6028/NIST.AI.100-1.

[5] C. Autio, R. Schwartz, J. Dunietz, S. Jain, M. Stanley, E. Tabassi, P. Hall, and K. Roberts, Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2024, doi: 10.6028/NIST.AI.600-1.

[6] European Parliament and Council, Regulation (EU) 2024/1689 of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act), Official Journal of the European Union, OJ L 2024/1689, 12 Jul. 2024.

[7] IEEE Standards Association, IEEE Std 7000-2021: IEEE Standard Model Process for Addressing Ethical Concerns during System Design. New York, NY, USA: IEEE, 2021, doi: 10.1109/IEEESTD.2021.9536679.

[8] J. Mokander and L. Floridi, "Operationalising AI governance through ethics-based auditing: an industry case study," AI and Ethics, vol. 3, no. 2, pp. 451-468, 2023, doi: 10.1007/s43681-022-00171-7.

[9] A. Taeihagh, "Governance of Generative AI," Policy and Society, vol. 44, no. 1, pp. 1-22, 2025, doi: 10.1093/polsoc/puaf001.

[10] A. Reuel and T. A. Undheim, "Generative AI Needs Adaptive Governance," arXiv:2406.04554, 2024, doi: 10.48550/arXiv.2406.04554.

[11] P. D. Koenig and G. Wenzelburger, "The legitimacy gap of algorithmic decision-making in the public sector: Why it arises and how to address it," Technology in Society, vol. 67, Art. no. 101688, 2021, doi: 10.1016/j.techsoc.2021.101688.

[12] M. Mitchell et al., "Model Cards for Model Reporting," in Proc. Conf. Fairness, Accountability, and Transparency (FAT* '19), Atlanta, GA, USA, 2019, pp. 220-229, doi: 10.1145/3287560.3287596.

[13] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," Computer, vol. 36, no. 1, pp. 41-50, 2003, doi: 10.1109/MC.2003.1160055.

[14] B. H. C. Cheng et al., "Software Engineering for Self-Adaptive Systems: A Research Roadmap," in Software Engineering for Self-Adaptive Systems, Berlin, Germany: Springer, 2009, pp. 1-26, doi: 10.1007/978-3-642-02161-9_1.

[15] Y. Brun et al., "Engineering Self-Adaptive Systems through Feedback Loops," in Software Engineering for Self-Adaptive Systems, Berlin, Germany: Springer, 2009, pp. 48-70, doi: 10.1007/978-3-642-02161-9_3.

[16] D. Garlan, B. Schmerl, and S.-W. Cheng, "Software Architecture-Based Self-Adaptation," in Autonomic Computing and Networking, New York, NY, USA: Springer, 2009, pp. 31-55, doi: 10.1007/978-0-387-89828-5_2.

[17] L. C. Siebert et al., "Meaningful human control: actionable properties for AI system development," AI and Ethics, vol. 3, no. 1, pp. 241-255, 2023, doi: 10.1007/s43681-022-00167-3.

[18] A. Desai, S. Ghosh, S. A. Seshia, N. Shankar, and A. Tiwari, "SOTER: A Runtime Assurance Framework for Programming Safe Robotics Systems," in Proc. 49th Annual IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN), 2019, pp. 138-150, doi: 10.1109/DSN.2019.00027.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:22:13 UTC from IEEE Xplore. Restrictions apply.